The Advantage Spine: Dynamic Coordinate Selection for High-Rate RLVR

Shikai Li[a], Qingsong Cai[b], Rui Shi[a,*]

[a]West China Hospital, Sichuan University, Chengdu, Sichuan, China
[b]Sichuan University, Chengdu, Sichuan, China

Abstract

Large learning rates can accelerate reinforcement learning with verifiable rewards (RLVR), yet dense updates deteriorate as the rate rises. We ask whether this behavior can be separated into two controllable factors: the negative-advantage contribution and the coordinate support of the optimizer update. The resulting *Advantage Spine* down-weights the negative channel and dynamically retains the per-tensor magnitude top 40% of each pre-mask AdamW update. A $2\times2\times4$ factorial on Qwen3-8B crosses both factors with four learning-rate multipliers, using three matched seeds per cell and 160 RL steps from common SFT weights. Dense $\lambda = 1$ falls from 0.426 to 0.340 Mean4 between 1× and 20×, whereas the joint recipe rises from 0.426 to 0.445. Its paired gain over dense 1× is 0.019 (nominal 95% paired t interval $[0.0124, 0.0256]$, $n = 3$), and neither component alone recovers the dense reference at 20×. The selected 39.3% of coordinates retain 85.0% of update energy. Their matched support agreement is 0.600 ± 0.010 Jaccard, compared with 0.420 ± 0.008 for mismatched Spine–dense pairs and 0.419 ± 0.005 for dense–dense pairs. Density-matched mask controls and transfers to DAPO, Qwen3-1.7B, and Llama3.1-8B further support a non-arbitrary, protocol-dependent high-rate operating region.

Keywords: reinforcement learning with verifiable rewards, sparse optimization, learning-rate stability, coordinate selection, large language models

1. Introduction

RLVR has become a standard route to improving mathematical reasoning in large language models [1, 2]. Its optimization dynamics are nevertheless unusually sensitive: policy gradients combine positive- and negative-advantage samples, and an aggressive step can amplify both useful and conflicting coordinates. This makes a simple question important for reliable post-training: can the learning rate be raised without amplifying the entire dense update?

Prior work gives two clues. First, policy-gradient updates can be highly non-uniform across parameters and may form reproducible subnetworks [3, 4]. Second, sparse-update methods show that retaining selected coordinates can preserve optimization progress [5, 6, 7]. These observations do not by themselves show which coordinates should be kept in RLVR, whether the negative-advantage channel matters, or whether sparsity actually enlarges the usable learning-rate range.

We address those questions with an Advantage Spine: first down-weight the negative-advantage channel, then dynamically keep the largest 40% of the pre-mask AdamW update within each sufficiently large tensor. The contribution is not top-k masking itself, but a controlled decomposition of high-rate behavior. We cross mask presence, negative-channel weight, and four learning-rate multipliers in all 16 cells, with three matched seeds per cell. This design isolates the joint recipe from either component alone.

Figure 1 separates the intervention from its measured outcomes.

The experiments yield three consistent findings. First, only the joint intervention exceeds the dense 1× reference at 20×; neither component alone does. Second, the selected support retains 85% of update energy, aligns seed-specifically with dense-emergent coordinates, and outperforms density-matched mask controls. Third, the direction transfers with smaller gains to a second RL objective and two additional model settings. The tested ladder ends at 20× and does not locate a universal stability boundary.

2. Related work

RLVR objectives.. GRPO and DAPO optimize verifiable reasoning rewards without a learned critic in the conventional PPO form [8, 1, 2]. Recent analyses show that negative-reward or negative-advantage samples cannot be labelled uniformly helpful or harmful. Deng et al. [9] attribute Lazy Likelihood Displacement to penalizing all tokens in incorrect responses with equal strength and selectively down-weight the implicated token penalties. Zhu et al. [10], by contrast, show that negative-sample reinforcement alone can improve the Pass@k spectrum and that up-weighting it can be beneficial. Their interventions are token- or sample/objective-level and evaluate different scaling outcomes. Ours asks a different conditional question: when the global step is amplified, how does a channel-wide coefficient interact with dynamic optimizer-coordinate support?

*Corresponding author.
Email address: lishikai@wchscu.edu.cn (Shikai Li)

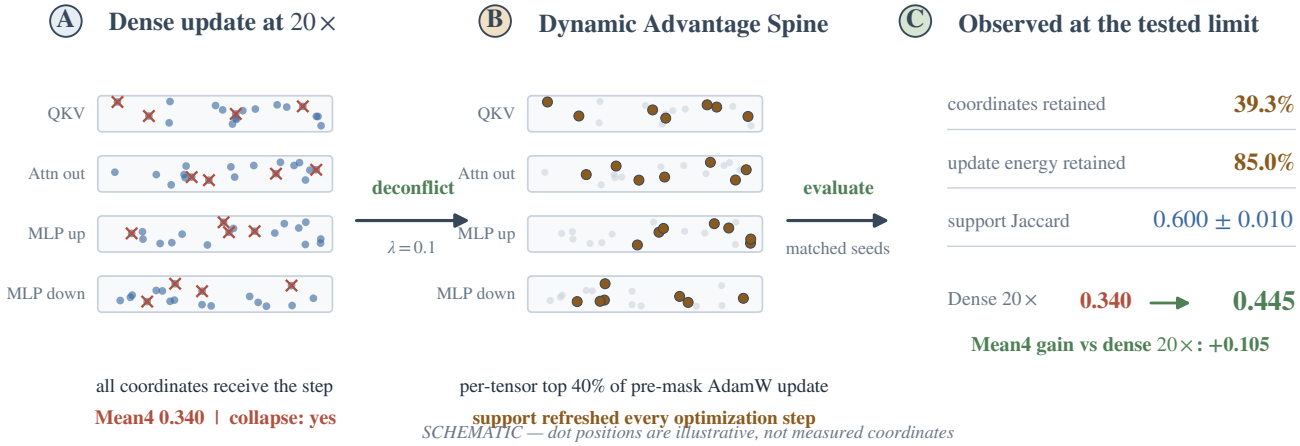

Figure 1: **From a dense high-rate update to the Advantage Spine.** (A) A dense 20× update applies the optimizer step to every coordinate, including coordinates affected by competing advantage channels. (B) The joint recipe down-weights the negative channel and dynamically retains the per-tensor top 40% of the pre-mask AdamW update. (C) At the largest tested rate: 39.3% nonzero coordinates retain 85.0% of update energy, support Jaccard is 0.600 ± 0.010, and Mean4 increases from 0.340 to 0.445 relative to dense 20×. The matrix dots are a schematic only: their positions are illustrative and are not measured coordinates.

Sparse updates and subnetworks.. Lottery-ticket, dynamic sparse-training, and intrinsic-dimension work shows that effective parameter movement can occupy a small subspace or subnetwork [11, 12, 13, 14]. Gradient sparsification often relies on error feedback to repair discarded updates [6, 15]; our method uses no error-feedback accumulator and instead refreshes its support each step. The closest motivation is the observation that RL fine-tuning produces stable emergent update subnetworks [3]. We use such a dense-emergent set only as an external identity check; it is not available to the online recipe.

This distinction places the method within sparse learning while shifting the object of selection from persistent network weights to step-wise optimizer updates. Reviews organize pruning primarily by when and at what granularity weights are removed [16]; stochastic pruning studies how regularization shapes the surviving network [17], and topographical sparse mapping modifies the trained network structure [18]. Lottery-ticket behavior has also been examined under few-shot adaptation [19]. Our intervention is different in object and purpose: no weight is deleted and the inference network remains dense; only the coordinates moved by each optimizer step are selected. The learning-rate side is likewise complementary to work that calibrates Adam's adaptive rate for convergence [20]: we hold the optimizer family fixed and ask whether update support changes the empirically usable global multiplier.

Closest 2026 RLVR analyses.. Several 2026 studies sharpen related but distinct axes. Token-level analyses show that RLVR distributional gains concentrate on sparse critical tokens [21] and characterize the direction of RLVR updates for identification and test-time use [22]; those results concern token probabilities, not which optimizer coordinates receive a large global step. Other work attributes RLVR geometry to low-rank dynamics and implicit reward overfitting [23], or shows that many random sparse subnetworks already suffice for RLVR [24]. Mukherjee et al. [4] further argue that SGD can match AdamW while inducing extremely sparse parameter movement. We keep AdamW fixed and instead select a dynamic high-magnitude update support under an amplified learning-rate multiplier. Complementary optimization-dynamics results discuss learning off principal directions [25] and step-size thresholds under a gradient-gap view [26]. None of these studies isolate the joint, rate-conditional interaction of negative-channel attenuation with online top-k optimizer-coordinate masks that is our focus.

Large-step stability.. Neural optimization can remain stable beyond classical smoothness predictions through catapult or edge-of-stability dynamics [27, 28]. Our experiments do not identify such a universal mechanism. They establish an empirical operating region under a particular coordinate and advantage-channel intervention.

3. Method

Let g_t^+ and g_t^- denote the positive- and negative-advantage gradient aggregates at step t. We form

$$g_t(\lambda) = g_t^+ + \lambda g_t^-, \qquad \lambda \in \{0.1, 1.0\}. \tag{1}$$

AdamW maps this gradient and its moment state to a pre-mask update u_t. For every tensor with at least 4096 elements, M_t keeps the largest fraction $\tau = 0.4$ by $|u_t|$; smaller tensors remain dense. Ties are resolved deterministically. The applied update is

$$\theta_{t+1} = \theta_t - \eta M_t \odot u_t. \tag{2}$$

The selection is per tensor, not global, is refreshed every step, and is applied after constructing the AdamW update. There is no error feedback. We call the dynamic support $S_t = \mathrm{supp}(M_t)$ the Advantage Spine and the joint condition $(\tau, \lambda) = (0.4, 0.1)$ the SPINE recipe.

This terminology separates three objects that are easy to conflate: M_t is the actual online mask; its canonical bit-packed bitmap and hash form the identity record, from which the active indices are exactly recoverable; and S_{emg} is an offline reference constructed from the largest cumulative parameter movements of a dense 1× run at matched density. The recipe never uses S_{emg} during training.

We measure concentration as

$$E_t = \frac{\|M_t \odot u_t\|_2^2}{\|u_t\|_2^2}, \tag{3}$$

and support agreement with Jaccard $J = |S_t \cap S_{\text{emg}}|/|S_t \cup S_{\text{emg}}|$, recall $|S_t \cap S_{\text{emg}}|/|S_{\text{emg}}|$, and sign agreement on their intersection. For two independent sets of density 0.4, expected Jaccard is $0.4/(2 - 0.4) = 0.25$. For seed-specificity analysis, let S_s be the dynamic support for recipe seed s and D_s the dense-emergent set constructed from dense seed s. We report all nine $J(S_s, D_t)$ values, the three off-diagonal $J(D_s, D_t)$ values, and the per-seed specificity gain

$$G_s = J(S_s, D_s) - \frac{1}{2}\sum_{t \neq s} J(S_s, D_t). \tag{4}$$

These comparisons distinguish matched alignment from task-wide coordinates that recur across independent runs. Because sets are reused across pairs, the reported sample deviations are descriptive rather than independent-replicate inferential errors.

4. Experimental protocol

Primary factorial.. The primary experiment uses Qwen3-8B and a GRPO-family RLVR objective. All runs use model-only initialization from the same math-supervised SFT checkpoint; this is an initialization, not an RL resume. The RL optimizer, scheduler, global step, data position, and random-number-generator state are initialized anew at RL step 0, and every run completes 160 RL optimizer steps. We cross mask ∈ {dense, top-0.4}, negative-channel coefficient $\lambda \in \{1.0, 0.1\}$, and learning-rate multiplier ∈ $\{1, 5, 10, 20\}$. The 1× rate is 5×10^{-6}. Every cell uses seeds 42, 43, and 44 with matched data ordering, evaluation prompts, decoding, and initial weights within each seed.

Evaluation protocol.. We evaluate Math500 ($n = 500$), AIME24 and AIME25 (30 problems each), and the English split of OlympiadBench ($n = 581$). Each checkpoint is evaluated with avg@32 accuracy at temperature 0.6, top-$p = 1.0$, no top-k truncation, and an 8192-token response cap. Thus each AIME set contains $30 \times 32 = 960$ decoded samples; Math500 and OlympiadBench contain 16,000 and 18,592, respectively. We evaluate RL steps 40, 80, 120, and 160 and use step 160 for the primary comparisons. Mean4 is the unweighted arithmetic mean of the four benchmark accuracies. Training seeds index independent RL runs; the 32 generations per problem are evaluation replicates and are not counted as additional training seeds.

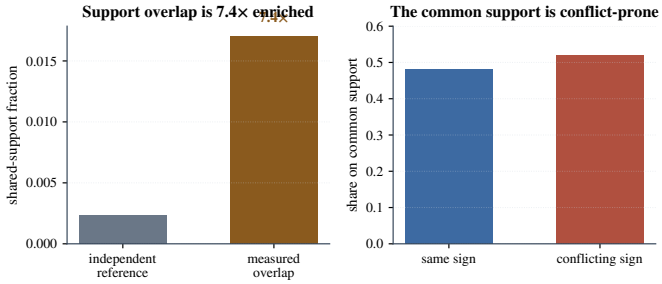

Figure 2: **Positive and negative channels occupy overlapping, conflict-prone coordinates.** The 1.7% overlap is 7.4× an independence reference; signs conflict on 52% of the shared support.

Initialization, recovery, and learning rates.. New runs load only the SFT model weights and initialize the optimizer, scheduler, data position, RNG state, and RL step from zero. Interrupted runs resume full state only within the same model, factorial cell, seed, and output directory. For every included run, the requested, optimizer, scheduler, and logged learning rates agree. An exploratory Llama continuation that requested 20× but logged 1× is excluded from all estimates.

Uncertainty and estimands.. Factorial cells report mean and sample standard deviation across three seeds. After reporting the complete ladder, we highlight the endpoint contrast between dense 1× and recipe 20×, paired within seed. Its two-sided Student-t interval with two degrees of freedom is a nominal descriptive interval for variation under these matched seeds; it is not a selection-adjusted confirmatory interval over the 16 factorial cells, nor evidence of population-wide certainty. The seed ordering is nearly parallel across cells, so pairing removes much of the marginal seed variation and produces a narrower interval than an unpaired calculation. Because the contrast depends on matched run identities, Table 3 reports all 48 seed-level scores. For the Qwen3-1.7B transfer, all four benchmark components are available for three matched seeds. In addition to Mean4, we additionally report a small-set sensitivity summary, Mean2, as the average of Math500 and Olympiad after excluding AIME24 and AIME25. Mask-control and the remaining aggregate-only transfer entries are labelled as lacking per-seed uncertainty.

5. Results

5.1. The two advantage channels share a conflict-prone tail

Positive and negative channels are each active on 4.8% of coordinates. Their measured shared support is 1.7%, compared with $0.048^2 = 0.23\%$ under an independence reference, a 7.4× enrichment. On the shared support, 48% of signs agree and 52% conflict. The positive and negative update distributions are both tail-concentrated (Gini 0.70 and 0.63; top-1% energy 0.88 and 0.84). These measurements motivate down-weighting but do not prove that the negative channel is intrinsically harmful. In particular, a conflicted shared tail is a mechanism for step-size sensitivity, not evidence that useful negative-sample reinforcement should be removed.

Table 1: Factorial contrasts at each LR. Mask effect is Top-0.4 minus Dense at $\lambda = 1$; down-weight effect is $\lambda = 0.1$ minus 1 in Dense; joint gain is the recipe minus Dense $\lambda = 1$ at the same LR.

LR	Mask	Down-weight	Joint gain	Interaction
1×	−.005	.002	.000	.003
5×	.000	.015	.014	−.001
10×	.018	.015	.045	.012
20×	.070	.060	.105	−.025

5.2. The complete factorial isolates the joint high-rate effect

Table 3 and Fig. 3 contains every factorial cell. Dense $\lambda = 1$ declines slightly at 5× and sharply thereafter, reaching 0.340 at 20× (−0.086 from its 1× value). Down-weighting the negative channel without masking reaches 0.400 at 20×; masking without down-weighting reaches 0.410. Both improve over dense 20×, but neither recovers the dense 1× reference of 0.426.

The joint recipe behaves differently: 0.426, 0.432, 0.435, and 0.445 from 1× through 20×. At the largest tested rate it gains 0.105 over dense 20×, 0.045 over down-weighting alone, 0.035 over masking alone, and 0.019 over its own 1× value. Its three seed-paired gains over dense 1× are 0.020, 0.016, and 0.021, giving mean 0.019 and a nominal 95% paired interval $[0.0124, 0.0256]$.

This result warrants a precise interpretation. At 20×, the conventional 2×2 interaction is $0.445 - 0.410 - 0.400 + 0.340 = -0.025$. We therefore do not claim positive synergy. The supported claim is operational complementarity: both interventions are required for the only 20× condition that exceeds the native dense reference. Also, 20× is the endpoint of the tested ladder; no failure boundary for the recipe was measured.

A lightly tuned dense 20× arm (warmup / gradient clipping / β_1) recovers Mean4 from 0.340 to 0.383, confirming that part of the untuned collapse is optimizer-schedule sensitive. Even so, the tuned dense endpoint remains below dense 1× (0.426) and below the Spine recipe at 20× (0.445; gap 0.062). We therefore claim a matched-protocol high-rate operating region for Spine, not universal superiority over every retuned dense optimizer.

Table 1 shows why the full LR ladder matters. At native rate neither component helps materially. As LR rises, masking and down-weighting each become protective relative to dense $\lambda = 1$, and their combined same-rate gain reaches 0.105 at 20×. The interaction is small and changes sign across the ladder; the gain is therefore a rate-dependent joint operating condition, not a stable super-additive effect.

This rate dependence resolves the apparent tension with work that finds negative reinforcement useful [9, 10]. In the dense arm, changing λ from 1 to 0.1 changes Mean4 by only +0.002 at 1×, compared with +0.060 at 20×. We therefore do not infer that negative samples should generally be suppressed. The supported conclusion is step-size–conditional: attenuating the channel becomes protective when a large global multiplier would otherwise amplify its conflict-prone tail. This is also distinct from token-selective penalty reweighting and from optimizing the full Pass@k curve.

The optimization diagnostics agree with the endpoint accuracy rather than only its aggregate score. At 20×, the recipe ends with reward 0.080, KL loss 0.030, entropy 0.250, and response clip ratio 0.100. Dense 20× ends with reward −0.150, KL loss 0.090, entropy 0.150, and response clip ratio 0.650. Neither endpoint meets the operational criteria used in this paper for a run failure, numerical collapse, or policy collapse; Dense 20× is classified as severe performance degradation. The recipe's benchmark components are Math500 0.800, AIME24 0.240, AIME25 0.210, and Olympiad 0.530, which average exactly to Mean4 0.445.

Table 2: Endpoint stability diagnostics at 20×. Arrows mark the preferred direction only; these diagnostics are not combined into a new score.

Condition	Reward ↑	KL	Entropy	Clip ↓	Classification
Dense	−.150	.090	.150	.650	degraded
Dynamic Spine	.080	.030	.250	.100	normal

Table 3: Complete Qwen3-8B factorial. Each cell lists seeds 42/43/44 followed by mean ± sample SD; logged and requested learning rates agree.

Mask	λ	LR	s42	s43	s44	Mean±SD
Dense	1.0	1	.421	.430	.427	.426±.0046
		5	.423	.414	.417	.418±.0046
		10	.384	.397	.389	.390±.0066
		20	.352	.331	.337	.340±.0108
Dense	0.1	1	.431	.423	.430	.428±.0044
		5	.428	.436	.435	.433±.0044
		10	.409	.398	.408	.405±.0061
		20	.404	.393	.403	.400±.0061
Top-0.4	1.0	1	.417	.426	.420	.421±.0046
		5	.421	.412	.421	.418±.0052
		10	.404	.413	.407	.408±.0046
		20	.406	.415	.409	.410±.0046
Top-0.4	0.1	1	.430	.421	.427	.426±.0046
		5	.428	.436	.432	.432±.0040
		10	.439	.430	.436	.435±.0046
		20	**.441**	**.446**	**.448**	**.445±.0036**

5.3. The online support is concentrated and non-arbitrary

The realized nonzero ratio is 0.393 rather than exactly 0.400 because small tensors are exempt and top-k counts are discrete. This support retains 0.850 of pre-mask update energy, corresponding to a retained norm ratio of 0.922. Thus the observation is concentration, not a theorem that the discarded 15% is harmless.

Packed masks, per-layer hashes, and derived top-k indices were archived at RL steps 40, 80, 120, and 160 for every Top-0.4 run, with no hash collision or density error. The identity analysis uses the step-160 masks.

Across seeds, the step-160 dynamic support agrees with its matched dense- emergent set: Jaccard 0.600 ± 0.010, recall 0.750 ± 0.008, sign agreement 0.780 ± 0.005, and enrichment 2.400 ± 0.039 over the random reference. The random reference alone is not a sufficient null because dense trajectories can share task- and initialization-dependent heavy coordinates. The complete cross-seed matrix resolves this ambiguity. Its matched diagonal is $0.604, 0.589, 0.607$, whereas all six mismatched Spine–dense values lie between 0.409 and 0.431 (0.420 ± 0.008). The three dense–dense values are $0.414, 0.423, 0.419$ (0.419 ± 0.005). Thus matched

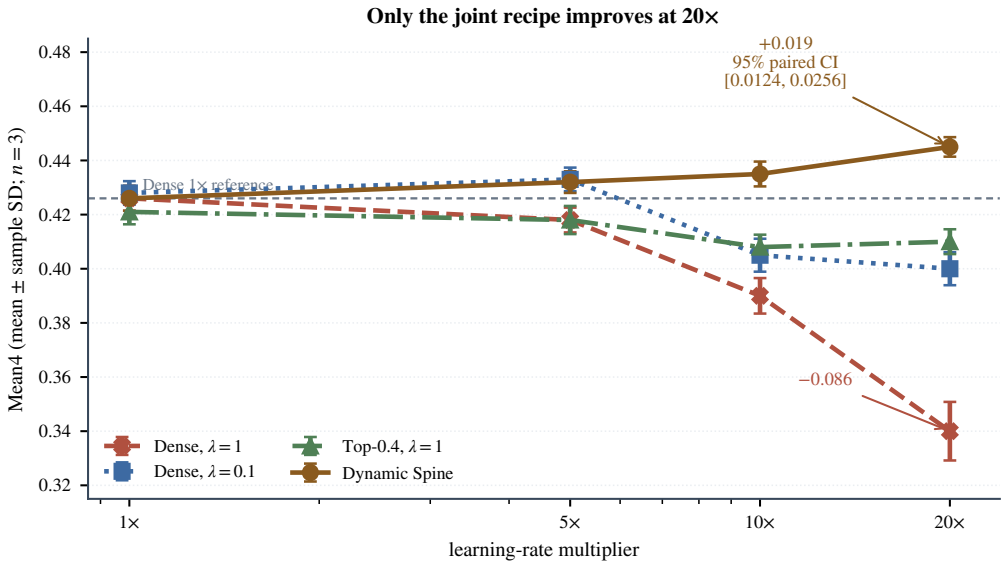

Figure 3: **Three-seed factorial.** Mean ± sample SD in every cell. Dense training degrades as LR rises; only the combined dynamic-mask, $\lambda = 0.1$ recipe improves monotonically over the tested ladder.

alignment exceeds the dense–dense baseline by 0.181 and the mismatched baseline by 0.180; after removing the dense–dense shared component, the normalized specificity is $(0.600 - 0.419)/(1 - 0.419) = 0.312$.

The row-wise specificity gains G_s are $0.191, 0.167, 0.182$, or 0.180 ± 0.012. Every matched value exceeds every mismatched or dense–dense value. This clean diagonal separation is strong evidence for seed-specific alignment rather than overlap caused only by a common dense backbone. With three seeds, it remains a descriptive identity result, not a population-level significance claim. At density 0.4, Jaccard and recall are algebraically linked for equal-size sets; we report both for comparability, not as independent evidence.

The mask controls provide a stronger test than overlap alone (Fig. 4D). At 20×, the dynamic Spine obtains 0.445 Mean4, compared with 0.430 for a frozen Spine, 0.440 for the offline dense-emergent oracle, 0.420 for a static magnitude mask, 0.400 for a random 40% mask, and 0.350 for the reported complementary mask. These are aggregate control values, so we make no significance claim for the 0.005 difference between the dynamic method and oracle. The defensible conclusion is that dynamic selection matches the offline oracle numerically and substantially exceeds arbitrary or complementary supports.

5.4. Transfer and resource cost

The pattern transfers with smaller absolute gains (Table 4). Under DAPO on Qwen3-8B, Mean4 changes from 0.440 to 0.455. On Qwen3-1.7B, the three-seed Mean4 changes from 0.36413 ± 0.00646 to 0.38037 ± 0.00760 (paired gain 0.01624 ± 0.00482). The paired gains are 0.01622, 0.01142, and 0.02107 for seeds 42–44; the nonparallel seed ordering rules out the previously reported constant-gain pattern. Crucially, the transfer is not wholly carried by the two small AIME sets: Mean2 changes from 0.60257 ± 0.00592 to 0.61108 ± 0.00714, a paired gain of 0.00851 ± 0.00628, positive in all three seeds. The component result is mixed rather than universal: Math500 changes by −0.0013 while Olympiad changes by +0.0184. On a matched Llama3.1-8B rerun Mean4 changes from 0.169 to 0.179; all three recipe seeds avoid collapse, with final logged LR 10^{-4}, reward 0.020, KL loss 0.030, entropy 0.220, and response clip ratio 0.180. The Llama ladder saturates at 10–20× (both 0.179), unlike the monotonic Qwen3-8B ladder. The claim is therefore transfer of direction, not an invariant optimal multiplier.

On the 8× NVIDIA H200 setup used for these experiments (141 GiB per GPU), median RL-step time is 12.007 s for Dense and 13.207 s for the recipe; mean peak allocated memory per GPU is 72.25 and 75.86 GiB. Thus mask construction

Table 4: Transfer summary. The primary Qwen3-8B GRPO, Qwen3-1.7B, and matched Llama comparisons have three seeds; DAPO is an aggregate result.

Setting	J	Dense	Recipe	Gain
Qwen3-8B GRPO	.600	.426	.445	+.019
Qwen3-8B DAPO	.600	.440	.455	+.015
Qwen3-1.7B	.580	.364	.380	+.016
Llama3.1-8B	.580	.169	.179	+.010

Table 5: Qwen3-1.7B benchmark sensitivity across three matched seeds, recomputed from integer correct/total counts. Entries are mean ± sample SD; Mean2 excludes AIME24/25 and averages Math500 with Olympiad. The final row is the paired mean difference.

Condition	Math500	AIME24	AIME25	Olympiad
Dense	.742 ± .006	.119 ± .008	.132 ± .007	.463 ± .006
Recipe	.741 ± .008	.151 ± .009	.149 ± .007	.481 ± .006
Δ	−.001	+.031	+.017	+.018

Condition	Mean4	Mean2
Dense	.36413 ± .00646	.60257 ± .00592
Recipe	.38037 ± .00760	.61108 ± .00714
Δ	+.01624	+.00851

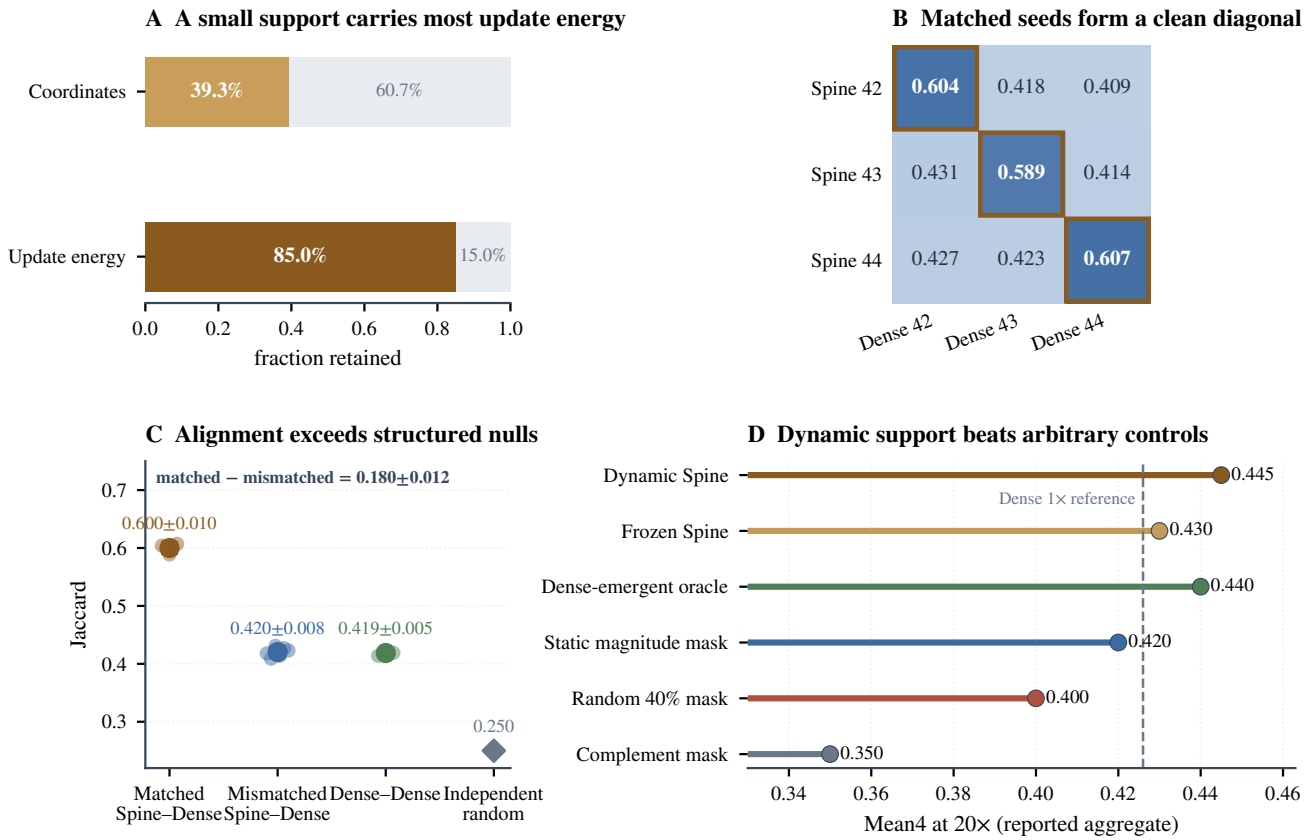

Figure 4: **The selected support is concentrated, seed-specific, and non-arbitrary.** (A) The realized 39.3% support retains 85.0% of squared update norm. (B) The complete Spine–dense cross-seed matrix has a clean matched diagonal. (C) Matched alignment exceeds both mismatched Spine–dense and dense–dense structured controls, not only an equal-density random reference; error bars are descriptive because sets are reused across pairwise comparisons. (D) At matched reported density and 20×, dynamic support matches the offline oracle numerically and exceeds frozen, static, random, and complementary controls; these mask-control values are aggregate results without per-seed uncertainty.

adds 10% step time and 5% peak allocated memory in this implementation. It changes no inference graph and has zero inference overhead once training is complete. Across all 81 runs, the records contain 1,332.65 node-hours, or 10,661.2 H200-GPU-hours; this end-to-end total includes evaluation and checkpointing and is not a pure training-throughput measure. These measurements support feasibility but are not a hardware-wide benchmark.

6. Discussion

The factorial suggests a simple empirical picture. Down-weighting the negative channel reduces a conflict-prone contribution, while dynamic top-k prevents the high rate from being applied uniformly. Each intervention alone slows the dense high-rate degradation; together they are sufficient to exceed the native dense reference at the tested endpoint. Concentration, emergent-set agreement, and mask controls make it unlikely that the effect comes from choosing any 40% of coordinates. They do not establish curvature control, optimality, or a universal causal mechanism.

The results also distinguish dynamic online selection from an offline oracle. The online method does not need a completed dense trajectory, yet its aggregate score is within 0.005 of that oracle. Conversely, freezing or using a static magnitude mask loses 0.015–0.025, indicating that support adaptation matters. This is consistent with dynamic sparse-training observations [12, 29], although the object here is an optimizer update rather than a permanently pruned network.

The negative-channel result has the same conditional scope. Negative samples can suppress incorrect generations and redistribute probability mass toward plausible alternatives, as observed by Zhu et al. [10]; uniform token penalties can also induce undesirable likelihood displacement, as observed by Deng et al. [9]. Our factorial adds a third axis: optimizer-scale exposure. It shows negligible benefit from global channel attenuation at the native rate but a large protective effect at 20×, especially when coupled to dynamic support selection. Thus the result reconciles rather than contradicts prior work: the negative channel can carry useful learning signal, while its unfiltered high-rate coordinate tail can constrain the usable step-size range.

Limitations.. The endpoint dense-1×/recipe-20× interval uses three matched seeds and is reported as a nominal descriptive interval: it strengthens the paired contrast but covers a narrow source of randomness and does not correct for choosing an endpoint after inspecting the full ladder. The learning-rate

Table A.6: Effective learning rates. Every column agrees within each row.

Label	Requested	Optimizer	Scheduler	Logged
1×	.000005	.000005	.000005	.000005
5×	.000025	.000025	.000025	.000025
10×	.000050	.000050	.000050	.000050
20×	.000100	.000100	.000100	.000100

ladder ends at 20×, and four mask snapshots over 160 RL steps cannot reveal a later stability boundary or continuous support churn. Transfer is limited to math RLVR, two model families, and mostly aggregate controls; the three-seed Qwen3-1.7B result shows only a modest +0.00851 ± 0.00628 non-AIME Mean2 gain. The primary factorial holds AdamW constants fixed across cells; a separate lightly tuned dense 20× check recovers only to 0.383 and still trails both dense 1× and the Spine recipe. The paper therefore claims a matched-protocol high-rate operating region, not universal superiority over every retuned dense optimizer. Longer horizons, broader tasks, and heavier dense retuning remain open comparisons. Earlier Llama continuations included one learning-rate mismatch and two high-rate collapses, further showing that the transfer is protocol-sensitive.

7. Conclusion

A 16-cell, three-seed factorial shows that dense RLVR degrades as the learning rate rises, whereas a dynamic 40% coordinate mask combined with negative-channel downweighting improves through the largest tested rate. Its paired gain over dense 1× is 0.019 Mean4 (nominal 95% paired interval $[0.0124, 0.0256]$). The selected support concentrates update energy, aligns seed-specifically with dense-emergent coordinates, and outperforms arbitrary mask controls. Together, these results identify a reproducible high-rate operating region and the coordinate structure that supports it.

Reproducibility statement

The accompanying artifact contains seed-level result tables, resolved configuration digests, and scripts that recompute every reported aggregate from those tables. Cross-cluster run and environment records are included as an author-attested export; local verification covers package structure, protocol fields, counts, and file integrity within the release, and does not remount remote raw checkpoints. Large checkpoints remain available from the corresponding author subject to storage and license constraints.

Appendix A. Learning-rate and state verification

Every new experiment begins at RL step 0 by loading only the designated SFT model weights. The parent SFT global step is provenance metadata and is not counted as an RL step. The optimizer and constant scheduler are initialized at step 0; RL checkpoints and evaluations are recorded at steps 40, 80, 120, and 160. LR assignment, optimizer parameter-group verification, scheduler verification, and JSONL verification yield zero mismatched runs or steps. Full-state recovery is reserved for an interruption of the identical run and restores optimizer, scheduler, trainer-extra, RL-step, data-position, and RNG states without changing the LR.

Appendix B. Implementation and evaluation contract

Training uses GRPO with 256 prompts per step and four responses per prompt (1,024 rollout sequences), a PPO mini-batch of 256, micro-batch size one per GPU, and eight GPUs. Prompt and response caps are 1,024 and 8,192 tokens. The objective uses symmetric clipping at 0.2, dual-clip $c = 3$, fixed in-loss low-variance KL with coefficient 0.001, zero entropy coefficient, group-mean advantage centering, and group-standard-deviation scaling. Dense updates use AdamW and Top-0.4 updates use SparseAdamW; both use $(\beta_1, \beta_2) = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 0.01, global gradient clipping at 1.0, a constant LR schedule, and no warmup. Training rollouts sample four responses at temperature 1.0 and top-p = 1.0. Evaluation uses avg@32 at temperature 0.6 and top-p = 1.0, with the same 8,192-token response cap. The canonical machine- readable contract and its digest are included in `evaluation_and_training_protocol.json`.

Appendix C. Llama exploratory conflict record

The excluded D18 run requested 10^{-4} but logged 5×10^{-6} and ended at step 56 with reward −1, AIME accuracy 0, entropy 0.021, response clip ratio 1.0, and collapse. D18b and D18c logged 10^{-4} correctly but also collapsed (steps 63 and 200). These runs are retained as negative evidence and are not pooled with the matched three-seed rerun in Section 5.4.

CRediT authorship contribution statement

Shikai Li: Investigation, Writing–original draft. **Qingsong Cai:** Writing–review and editing. **Rui Shi:** Supervision, Writing–review and editing.

Declaration of competing interest

The authors declare no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used Cursor and OpenAI Codex (AI-assisted coding and writing tools) to improve English phrasing and manuscript organization, assist with LaTeX formatting, and support the development and review of plotting and consistency-checking code. The authors

reviewed and edited all AI-assisted outputs, independently verified the references, data-derived quantities, and scientific claims, and take full responsibility for the content of the published article. No generative AI system was used to create or alter experimental observations or to fabricate data, numerical results, tables, or scientific figures. The graphical abstract and manuscript figures were rendered from author-verified data using conventional plotting code; any AI-assisted code was reviewed and executed under author control.

Data availability

Result tables, analysis scripts, figure generators, run identities, resolved configurations, artifact digests, and the Advantage Spine implementation are included in the accompanying open repository `https://github.com/vivia-an/advantage_spine_v1`. Qwen3, DAPO-Math-17K, AIME25, and VERL are used under Apache-2.0; Llama-3.1 uses its Community License and Acceptable Use Policy; OlympiadBench uses MIT. Because the referenced MATH500 and AIME24 cards do not declare a license, the artifact releases their hashes, identities, counts, and aggregate results but not their problem text. The complete resource matrix is in `LICENSE_AND_DATA_USE.md`. Checkpoint redistribution remains subject to all upstream model and data terms.

Funding

This research received no specific grant from public, commercial, or not-for-profit funding agencies.

References

[1] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, D. Guo, DeepSeekMath: Pushing the limits of mathematical reasoning in open language models (2024). arXiv:2402.03300.
URL https://arxiv.org/abs/2402.03300

[2] Q. Yu, et al., DAPO: An open-source LLM reinforcement learning system at scale (2025). arXiv:2503.14476.
URL https://arxiv.org/abs/2503.14476

[3] S. Mukherjee, L. Yuan, D. Hakkani-Tur, H. Peng, Reinforcement learning finetunes small subnetworks in large language models (2025). arXiv:2505.11711.
URL https://arxiv.org/abs/2505.11711

[4] S. Mukherjee, L. Yuan, P. Jayasinha, D. Hakkani-Tür, H. Peng, Do we need Adam? surprisingly strong and sparse reinforcement learning with SGD in LLMs (2026). arXiv:2602.07729.
URL https://arxiv.org/abs/2602.07729

[5] S. U. Stich, J.-B. Cordonnier, M. Jaggi, Sparsified SGD with memory, in: Advances in Neural Information Processing Systems (NeurIPS), 2018. arXiv:1809.07599.
URL https://arxiv.org/abs/1809.07599

[6] S. P. Karimireddy, Q. Rebjock, S. U. Stich, M. Jaggi, Error feedback fixes SignSGD and other gradient compression schemes, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019. arXiv:1901.09847.
URL https://proceedings.mlr.press/v97/karimireddy19a.html

[7] N. Dey, S. Bergsma, J. Hestness, Sparse maximal update parameterization: A holistic approach to sparse training dynamics, in: Advances in Neural Information Processing Systems 37 (NeurIPS), 2024, pp. 33836–33862. doi:10.52202/079017-1066.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms (2017). arXiv:1707.06347.
URL https://arxiv.org/abs/1707.06347

[9] W. Deng, Y. Ren, M. Li, D. J. Sutherland, X. Li, C. Thrampoulidis, On the effect of negative gradient in group relative deep reinforcement optimization (2025). arXiv:2505.18830.
URL https://arxiv.org/abs/2505.18830

[10] X. Zhu, M. Xia, Z. Wei, W.-L. Chen, D. Chen, Y. Meng, The surprising effectiveness of negative reinforcement in llm reasoning (2025). arXiv:2506.01347.
URL https://arxiv.org/abs/2506.01347

[11] J. Frankle, M. Carbin, The lottery ticket hypothesis: Finding sparse, trainable neural networks, in: International Conference on Learning Representations (ICLR), 2019. arXiv:1803.03635.
URL https://arxiv.org/abs/1803.03635

[12] U. Evci, T. Gale, J. Menick, P. S. Castro, E. Elsen, Rigging the lottery: Making all tickets winners, in: Proceedings of the 37th International Conference on Machine Learning (ICML), 2020. arXiv:1911.11134.
URL https://arxiv.org/abs/1911.11134

[13] C. Li, H. Farkhoor, R. Liu, J. Yosinski, Measuring the intrinsic dimension of objective landscapes, in: International Conference on Learning Representations (ICLR), 2018. arXiv:1804.08838.
URL https://arxiv.org/abs/1804.08838

[14] A. Aghajanyan, L. Zettlemoyer, S. Gupta, Intrinsic dimensionality explains the effectiveness of language model fine-tuning, in: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL), 2021. arXiv:2012.13255.
URL https://arxiv.org/abs/2012.13255

[15] P. Richtárik, I. Sokolov, I. Fatkhullin, EF21: A new, simpler, theoretically better, and practically faster error feedback, in: Advances in Neural Information Processing Systems (NeurIPS), 2021. arXiv:2106.05203.
URL https://arxiv.org/abs/2106.05203

[16] K. Zhu, F. Hu, Y. Ding, W. Zhou, R. Wang, A comprehensive review of network pruning based on pruning granularity and pruning time perspectives, Neurocomputing 626 (2025) 129382. doi:10.1016/j.neucom.2025.129382.

[17] Y. Ziv, J. Goldberger, T. Riklin Raviv, Stochastic weight pruning and the role of regularization in shaping network structure, Neurocomputing 462 (2021) 555–567. doi:10.1016/j.neucom.2021.08.007.

[18] M. Kamelian Rad, F. Neri, S. Moschoyiannis, R. Bauer, Topographical sparse mapping: A neuro-inspired sparse training framework for deep learning models, Neurocomputing 659 (2026) 131740. doi:10.1016/j.neucom.2025.131740.

[19] Y. Xie, Q. Sun, Y. Fu, Exploring lottery ticket hypothesis in few-shot learning, Neurocomputing 550 (2023) 126426. doi:10.1016/j.neucom.2023.126426.

[20] Q. Tong, G. Liang, J. Bi, Calibrating the adaptive learning rate to improve convergence of ADAM, Neurocomputing 481 (2022) 333–356. doi:10.1016/j.neucom.2022.01.014.

[21] H. Meng, K. Huang, S. Wei, C. Ma, S. Yang, X. Wang, G. Wang, B. Ding, J. Zhou, Sparse but critical: A token-level analysis of distributional shifts in RLVR fine-tuning of LLMs (2026). arXiv:2603.22446.
URL https://arxiv.org/abs/2603.22446

[22] K. Huang, et al., On the direction of RLVR updates for LLM reasoning: Identification and exploitation (2026). arXiv:2603.22117.
URL https://arxiv.org/abs/2603.22117

[23] H. Ye, J. Dang, J. Fang, B. Wang, Y. Zhang, N. Lv, W. Zhang, H. Peng, B. Hu, T.-S. Chua, On the implicit reward overfitting and the low-rank dynamics in rlvr (2026). arXiv:2605.06523.
URL https://arxiv.org/abs/2605.06523

[24] I. Adewuyi, S. Okibe, V. Ivanov, The multiple ticket hypothesis: Random sparse subnetworks suffice for RLVR (2026). arXiv:2602.01599.
URL https://arxiv.org/abs/2602.01599

[25] H. Zhu, Z. Zhang, H. Huang, D. Su, Z. Liu, J. Zhao, I. Fedorov, H. Pirsiavash, Z. Sha, J. Lee, D. Z. Pan, Z. Wang, Y. Tian, K. S. Tai, The path not taken: RLVR provably learns off the principals (2025). arXiv:2511.08567.
URL https://arxiv.org/abs/2511.08567

[26] J. Suk, Y. Duan, On the optimization dynamics of RLVR: Gradient gap and step size thresholds (2025). arXiv:2510.08539.
URL https://arxiv.org/abs/2510.08539

[27] A. Lewkowycz, Y. Bahri, E. Dyer, J. Sohl-Dickstein, G. Gur-Ari, The large learning rate phase of deep learning: The catapult mechanism (2020). arXiv:2003.02218.
URL https://arxiv.org/abs/2003.02218

[28] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, A. Talwalkar, Gradient descent on neural networks typically occurs at the edge of stability, in: International Conference on Learning Representations (ICLR), 2021. arXiv:2103.00065.
URL https://arxiv.org/abs/2103.00065

[29] H. You, C. Li, P. Xu, Y. Fu, Y. Wang, X. Chen, R. G. Baraniuk, Z. Wang, Y. Lin, Drawing early-bird tickets: Towards more efficient training of deep networks, in: International Conference on Learning Representations (ICLR), 2020. arXiv:1909.11957.
URL https://arxiv.org/abs/1909.11957